SET

Autonomous multi-agent orchestration for Claude Code

Give it a spec. Get merged features.

Greenfield or brownfield. Full app or single module.

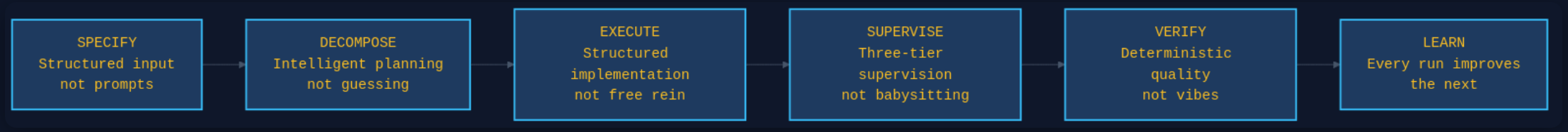
The problem with AI coding today

> **Same prompt, different result.** Run it twice, get two different applications.
> Agents hallucinate features, skip requirements, and forget everything between sessions.

We tried "just prompting" agents to build apps. Results:

- 3 agents chose **3 different table libraries** for the same spec
- Cart feature shipped **without price calculation** (no one asked for it

Six pillars -- what makes it work



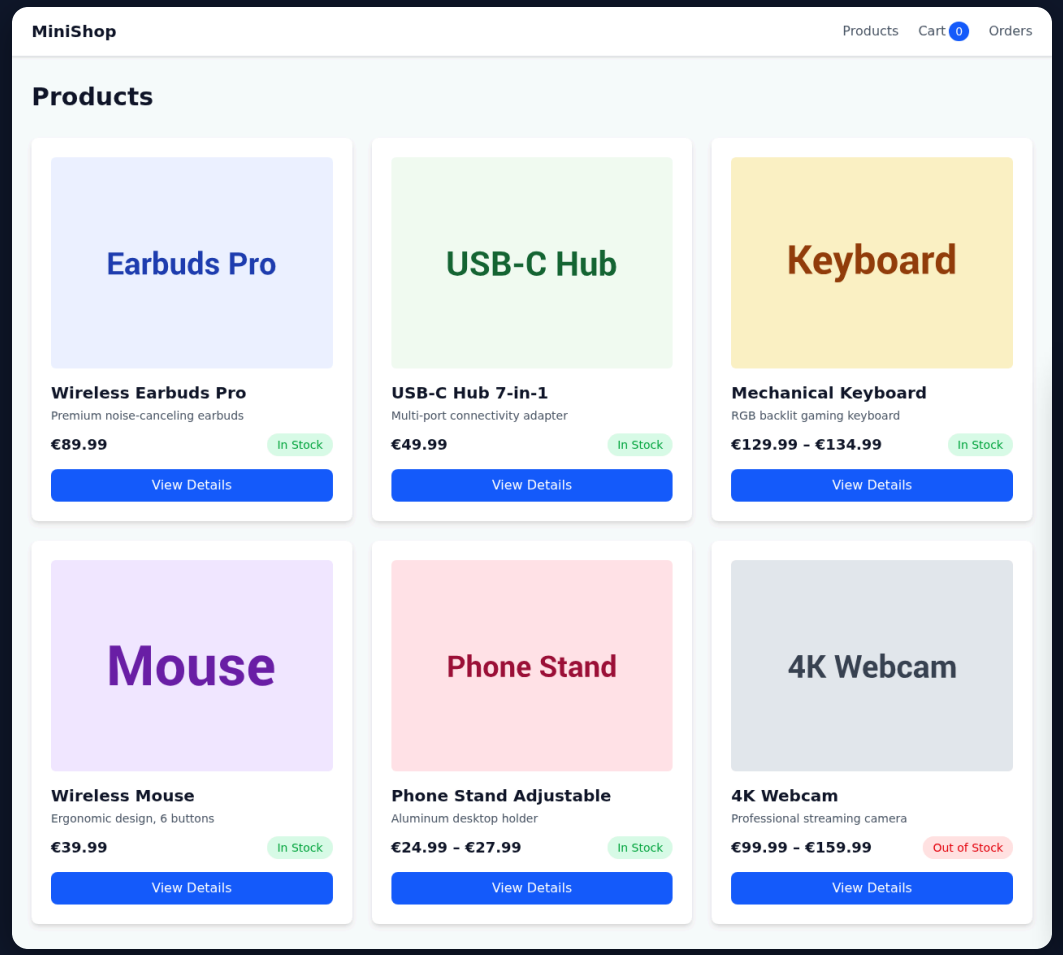
Pillar	In one sentence
SPECIFY	Structured input with requirements + acceptance criteria, not vague prompts
DECOMPOSE	Intelligent planning into parallel changes with dependency DAG
EXECUTE	Iterative implementation in isolated git worktrees, not single-shot prompts
SUPERVISE	Three-tier supervision (sentinel > orchestrator > agents), 30s crash recovery
VERIFY	7 deterministic gates -- exit codes, not LLM vibes

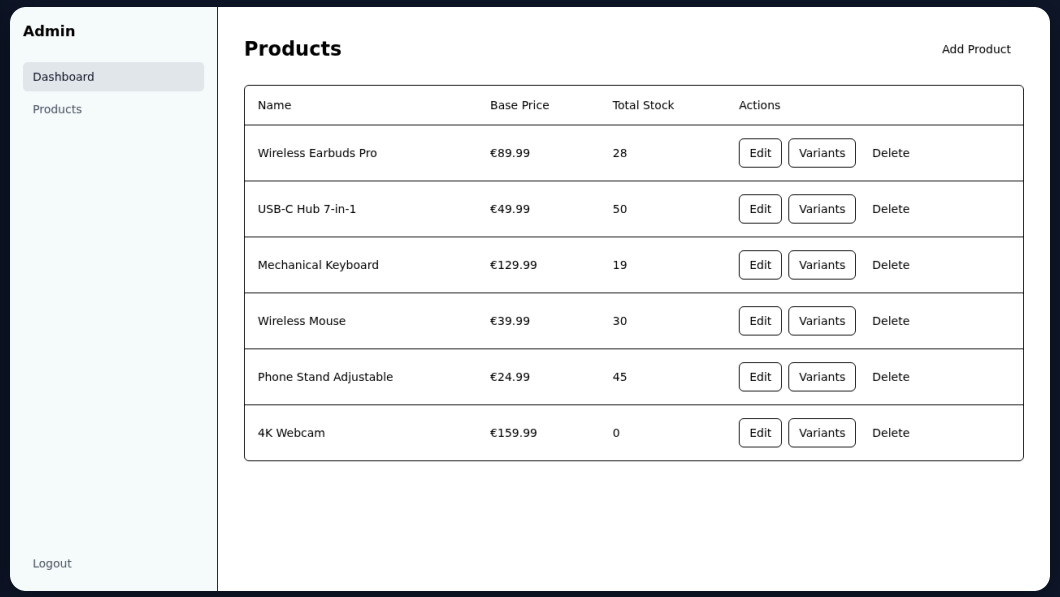
The pipeline

spec.md is the input. The system digests it into requirements, decomposes into
independent changes, dispatches parallel agents, verifies each through 7 gates,
merges to main, and replans if coverage is under 100%.

Fully autonomous. The sentinel supervises everything -- crashes, stalls, budget.

What it builds -- MiniShop





A complete **Next.js e-commerce app** -- product listing, cart, checkout,

The numbers

6/6
changes merged

1h 45m
wall clock

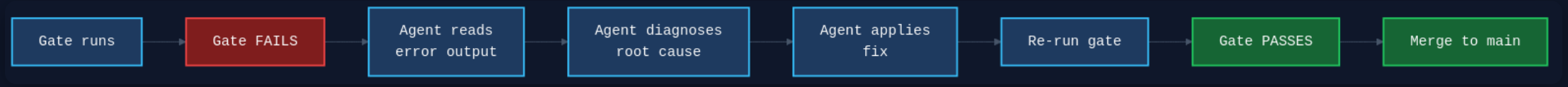
0
interventions

70
tests written

2.7M
tokens total

What	How
5 gate failures	All self-healed -- agent reads error, fixes, re-runs gate
83-87% convergence	Run same spec twice, measure structural overlap
100% schema match	Data model is fully deterministic across runs

Self-healing -- it doesn't retry, it investigates



Real example from MiniShop: Playwright auth test expected redirect to `/login`,
but middleware redirected to `/admin/login`. The gate caught it, the agent read the error, traced the middleware, updated 3 test files. No human involved.

The system fixed a bug in SET's own code during an E2E run -- and committed the fix

Works everywhere

	Greenfield	Brownfield	Isolated unit
What	Full app from spec + design	Your existing codebase	One module, one feature
Example	MiniShop: 6/6 merged, 1h 45m	SET builds itself: 1,500+ commits	"Add 3 API endpoints with auth"
Pipeline	6-15 parallel changes	Same gates, reads existing code	Single change, full verification

The entry barrier is not a 30-page spec. It can be a single task description.
The pipeline scales from a sentence to a specification.

Scale -- from simple to complex

	micro-web	MiniShop	CraftBrew
Complexity	4 pages	E-commerce with admin	Multi-tenant, 28 DB tables
Changes	3	6	15
Wall time	~20 min	1h 45m	~6h
Tokens	~50K	2.7M	~11M
Merge conflicts	0	0	4 (auto-resolved)
Result	100%	100%	100% (run #7)

Live demo

Dashboard | Scaffolds | Pipeline in action